Dimensionality reduction and unsupervised learning

Olivetti faces · four hundred photographs, no labels

LECTURE 6 | GÉRON CH. 7 & 8

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

Where we are

Five lectures, and every one of them was handed the answers:

Lecture	What the data came with
1–2 · California housing	a price for every district
3 · MNIST	a digit for every image
4 · Titanic	a survivor flag for every passenger
5 · CoverType	a cover type for every plot
6 · Olivetti faces	**✗ nothing**

Every model so far was scored against a label. Today there is no target column. That changes what a metric can be, what a baseline is, and how you find out you were wrong.

You have met PCA, k-means and DBSCAN as methods. Today is about where they are used, which one a task calls for, and what goes wrong in practice.

Today

1. **What unsupervised learning is**, and why anyone pays for it
2. **Four tasks from practice** — and for each, the one tool that fits and why the others are the wrong choice
3. **PCA on a worked example** — what it keeps and what it drops
4. **k-means** — choosing k with no labels, and looking at what you got
5. **DBSCAN and anomalies**, and where to spend forty labels
6. **What to look out for** when you do this for real
7. **Design exercise** — a grocery-delivery app's customer segments for its marketing team, in nine steps, with the pitfalls and how to address them

The derivations are in the notes. The slides are about the decisions.

Open Lecture 6 in Colab

What unsupervised learning is

Supervised

- You are given $(\mathbf{x}^{(i)}, y^{(i)})$
- The metric compares prediction to truth
- The baseline is “predict the majority”
- You know when you are wrong

Unsupervised

- You are given $\mathbf{x}^{(i)}$ only
- The metric compares the grouping to *itself*
- The baseline is “group at random”
- You find out you were wrong when someone looks

THE CONSEQUENCE FOR THE REST OF THE LECTURE

Every number computed today is a statement about **geometry**, not about people, customers or transactions. Whether the geometry *is* the thing you care about is a separate question, and it is answered by looking.

Why anyone pays for it

Outside a benchmark, data is cheap and labels are not. Every company has far more rows than it has verdicts about them. Three jobs recur, and every unsupervised method you know does one of them:

The job	What it buys	The methods
Find the groups	a handful of segments to act on, one decision per group instead of one per row	k-means, DBSCAN
Fewer numbers per item	a model that trains faster, a plot you can look at, a store that fits	PCA, the SVD
Flag the unusual	the few rows unlike all the rest, before anyone has labelled a single one	DBSCAN's noise, a reconstruction error

✓ And a fourth, quieter use: structure found without labels tells you *which* few rows are worth paying to label. The worked example ends there.

One — a retailer's customer segments: k-means

The marketing team wants a handful of segments, one campaign each, and *every* account placed in one. The rows are spend, order frequency, basket mix, time since the last order.

Tool	On this task
k-means ✓	a few compact groups, every account assigned, k small because a person has to write a campaign per segment — k is a business decision, and that is what k-means asks for
✗ DBSCAN	behaviour data has no empty space between groups, so either most accounts are noise or everything is one cluster; and the team cannot send a campaign to "noise"
✗ PCA / SVD	gives axes, not groups; a direction of spend is a description, not a segment anyone can write to

LOOK OUT FOR

The columns are in different units, and k-means only sees distance: scale first, or the segments are "spend" and nothing else.

Two — a payments app's unusual transactions: DBSCAN

The fraud team wants the few transactions unlike all the rest, every night, with no labels yet — the disputes that would label them arrive weeks later.

Tool	On this task
DBSCAN ✓	normal behaviour is dense and of any shape — commuters, weekend shoppers, subscriptions — and the output wanted is exactly its **noise** label: points no dense region claims, with no k to choose
✗ k-means	every transaction must belong to a cluster, so an anomaly becomes a far-off member of the nearest one and drags its centroid towards itself
✗ PCA	a reconstruction error only catches what leaves the main subspace; a fraudulent transaction that sits in a hole *inside* it scores as normal

LOOK OUT FOR

`eps` and `min_samples` together set the density a cluster must reach; the fraction called

Three — embeddings into a smaller model, and onto a plot: PCA

A search team stores a long vector per product and per query. The downstream model trains too slowly, the index is too large, and nobody can look at the data.

Tool	On this task
PCA ✓	a linear map to fewer numbers that keeps the directions where the vectors differ most, and states the price before you choose how many: the explained variance ratio
✗ k-means	replaces every vector by its centroid, so everything that distinguished two products in the same cluster is gone — quantisation, not compression
✗ DBSCAN	compresses nothing: it returns a label per row and keeps every column

LOOK OUT FOR

The two-dimensional plot is a projection, and distances on it are not distances in the data. And variance kept is not usefulness kept: read what the kept directions are.

Four — grouping unlabelled support tickets: the SVD

A support team has a backlog of tickets and no categories yet. Each ticket is a sparse row of word counts: thousands of columns, almost all zero.

Tool	On this task
Truncated SVD ✓	works on the sparse matrix as it is, and its leading directions are the recurring *themes* — the words that occur together; then group the tickets in that small space
✗ PCA	centres first, which turns a sparse matrix into a dense one that does not fit in memory, for a mean that is mostly zeros
✗ k-means or DBSCAN on the counts	in thousands of dimensions every ticket is about equally far from every other: k-means groups by ticket length and the commonest words, DBSCAN finds no dense region and calls everything noise

LOOK OUT FOR

The themes are read by a person; a direction that mixes two topics is not fixed by any amount of tuning, only by looking at its top words.

The tool follows the shape of the answer

What the consumer needs	The tool	What it assumes
every item in one of a few compact groups	**k-means**	convex, roughly equal groups; k given
dense regions of any shape, and a “none of these”	**DBSCAN**	a density that separates groups from the rest
fewer numbers per item, with the price stated	**PCA**	the useful variation is where the variance is
themes in a sparse table of counts	**truncated SVD**	the same, without centring

✓ Choose by what the consumer of the output can do with it. A segment nobody can write to, a noise label nobody can act on and a component nobody can read are all technically correct and all useless.

The rest of the lecture works one example in full, because every one of these tools has the same failure mode: it does exactly what it was asked, in a geometry that may not be the one you meant.

The worked example

The brief

> **THE REQUEST**
>
> “We have a photographic archive. We do not know how many distinct people are in it. Group the photographs by person, so that a human can name a *group* instead of naming every photograph.”

- The archive is not annotated, and nobody remembers who is in it
- A trained annotator costs money and time per photograph
- We are funded for **forty** labels. That is all

The same shape as the four examples: find the groups, compress to make it feasible, flag what does not belong, and spend the few labels well.

The corpus, and a photograph as a point

Quantity	Value
Photographs	400
Distinct people	40
Photographs per person	10
Pixels per photograph	4,096
The whole corpus in memory	6.6 MB

$$\mathbf{x}^{(i)} \in \mathbb{R}^{4096}$$

$$x_j \in [0, 1]$$

Flatten the 64 × 64 grid and a photograph is a vector. Every method today measures a distance between two of them.

✗ **Ten times more dimensions than points. Four hundred points cannot fill four thousand dimensions, and that is the first reason to compress before clustering.**

Forty people

One photograph of each of the 40 people. The corpus holds 400, and none of them arrives labelled



Same crop, same lighting rig, same background. What varies between rows is the person — and, as much, the lamp.

`y` exists. The brief does not have it

Olivetti is a benchmark, so it ships with the identity of every photograph. The archive we are pretending to hold does not.

> **THE RULE, KEPT FOR THE WHOLE LECTURE**
>
> `y` is used in exactly two ways, both announced on the slide where they happen: as the *forty labels we paid for*, and as an **audit**, to find out what those forty could not have told us.

Any other use of `y` is the same error as scoring on the test set, wearing different clothes. In practice you will have no `y` at all — which is why the audit slides matter: they show what you would not have been able to see.

Before anything is fitted: can this work at all?

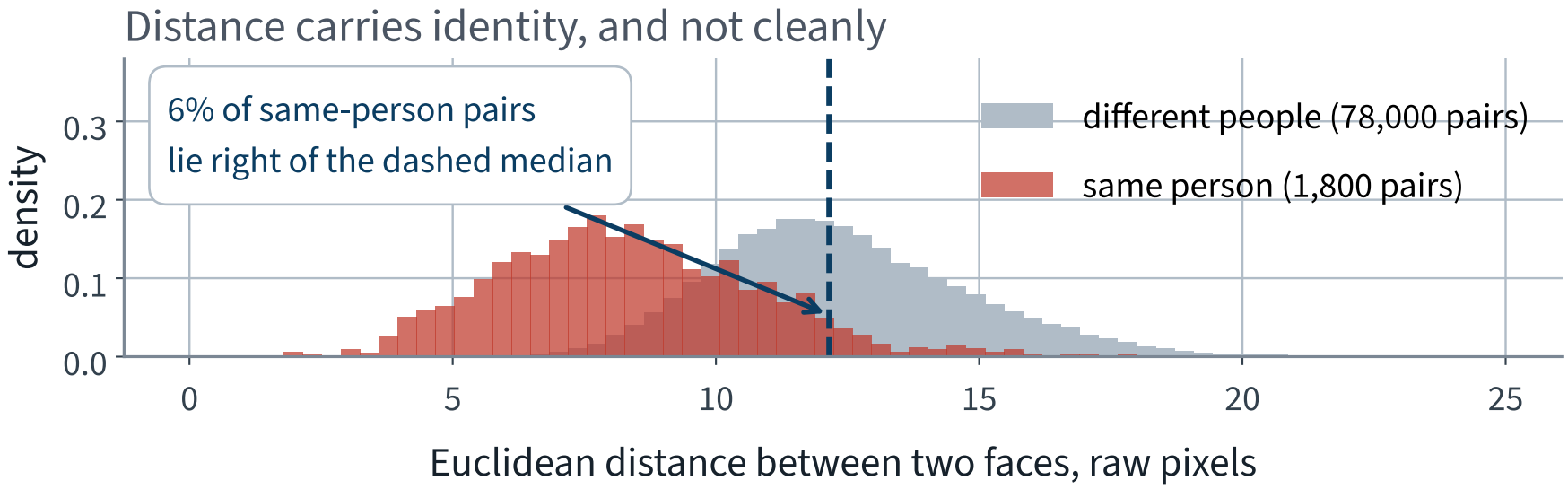
k-means groups points that are close in Euclidean distance. So one measurement decides whether the rest of the lecture can work (an audit — it uses the hidden labels):

Are two photographs of the same person closer than two photographs of different people?

Euclidean distance, raw pixels	Pairs	Mean	Median
Same person	1,800	8.38	8.15
Different people	78,000	12.41	12.15

✓ In practice: before you fit anything, measure whether the distance you are about to cluster on tracks the thing you care about. One cell, and it tells you whether you are about to tune or about to hope.

The gap is real, and it is small



✗ 6% of same-person pairs sit further apart than the *median* different-person pair. Pixel distance is partly identity and partly lighting — and whatever varies most owns the distance.

PCA: what it minimises, and what the SVD gives you

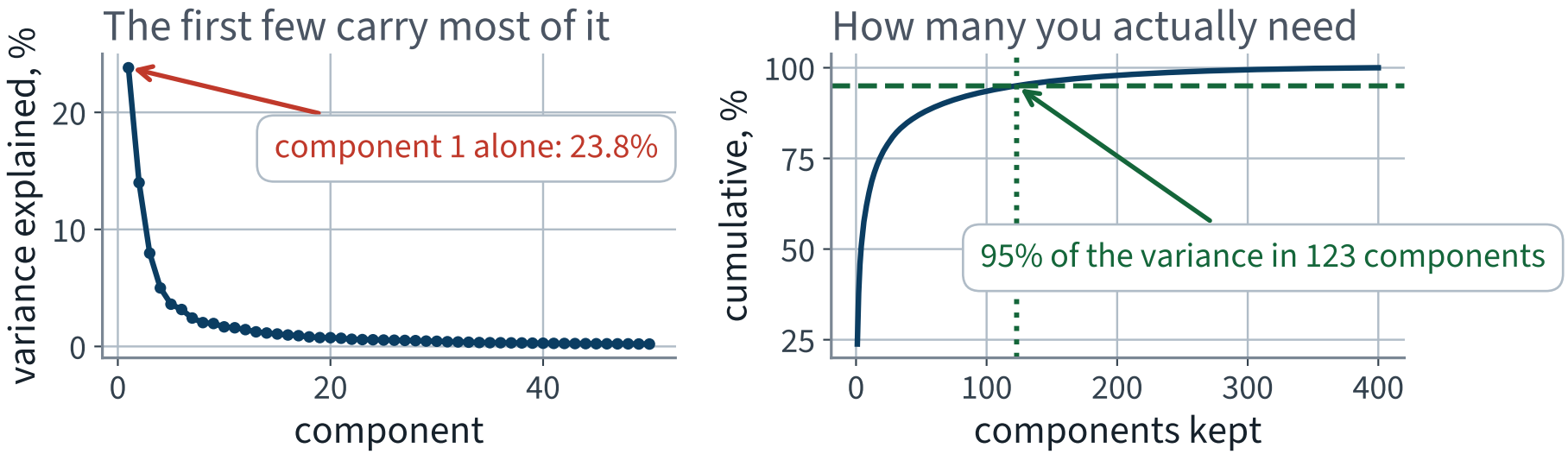
Centre the data by subtracting the mean face, giving $\tilde{\mathbf{X}}$; among all d-dimensional subspaces, PCA picks the one that loses least when the points are projected onto it:

$$\min_{\mathbf{W}^\top \mathbf{W} = \mathbf{I}_d} \left\| \tilde{\mathbf{X}} - \tilde{\mathbf{X}} \mathbf{W} \mathbf{W}^\top \right\|_F^2 = \sum_{j > d} \sigma_j^2 \qquad \text{at } \mathbf{W} = \mathbf{V}_d$$

- The answer is the SVD of the centred data, $\tilde{\mathbf{X}} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$: the first d right singular vectors. The error left is the tail of the spectrum, readable *before* you choose d
- The **explained variance ratio** $r_j = \sigma_j^2 / \sum_i \sigma_i^2$ is that statement normalised: keeping d components removes exactly $\sum_{j \le d} r_j$ of the squared error. The seven-line derivation is in the notes

✗ Centre first. `PCA` does it for you; a hand-rolled SVD does not, and the uncentred first component is the mean face — a plausible picture and no error message.

How many components you actually need



123 components hold 95% of the variance of 4,096 pixels — a factor of 33 fewer numbers per photograph, and the price of the cut is read off the right-hand panel before anything downstream is run.

In one line of code

```python
from sklearn.decomposition import PCA

Z = PCA(n_components=0.95, random_state=42).fit_transform(X)
print(Z.shape)                                  # (400, 123)
```

`n_components=0.95` — a float between 0 and 1 is read as a **variance target** rather than a component count, and scikit-learn chooses d for you from the cumulative ratio. Here it chooses 123.

✓ An integer means "this many components"; a float means "this much variance". One character apart, and `n_components=1` and `n_components=1.0` do completely different things. It is the first thing to check in a pipeline you inherit.

The mean face, then the first fifteen components

The mean face, then the first 15 principal components. Each component is itself an image



✗ A component is a vector in $\mathbb{R}^{4096}$, so it is an image — an ***eigenface***. Look at the first few. They are lighting — exactly what the distance histogram predicted would dominate. Variance kept is not usefulness kept.

The same held-out face, with components taken away

One held-out face, rebuilt from this many components



Recognisable well before 123 components. Identity survives a compression that the pixel values do not.

k-means, in five lines

1. Choose k centroids (scikit-learn uses *k-means++*, which spreads the initial ones out)
2. Assign every point to its nearest centroid
3. Move every centroid to the mean of its assigned points
4. Repeat 2–3 until no assignment changes
5. Repeat the whole thing `n_init` times and keep the best

> Step 5 is not optional. Lloyd's algorithm converges — to a local minimum that depends entirely on where you started. `n_init` changed its default between scikit-learn versions, so name it — and name `random_state`, or the segments change every time the job runs.

What k-means assumes, whether you asked or not

FOUR ASSUMPTIONS, THREE OF THEM FALSE HERE

Convexity. The partition is a Voronoi diagram of the centroids, so every cluster is convex.

Spherical, equal-sized. The only quantity is distance to a centre.

Total assignment. Every point belongs to exactly one cluster; there is no “none of these”.

k is given. Nothing in the algorithm chooses it.

The first three are what DBSCAN drops, and why it was the tool for the transactions. The fourth — choosing k — is what the next slides are about, because in practice nobody tells you k.

Definition: inertia

$$J(k) = \sum_{i=1}^{m} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}_{c(i)} \right\|_2^2$$

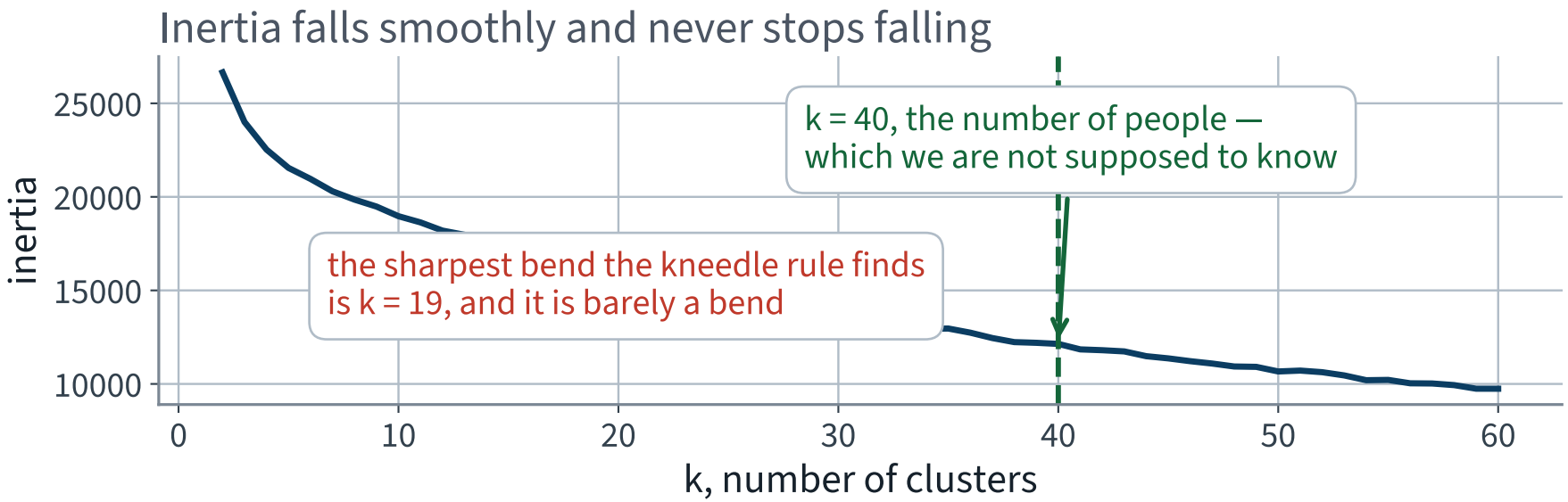
THE SUM OF SQUARED DISTANCES FROM EVERY POINT TO ITS ASSIGNED CENTROID — WHAT K-MEANS MINIMISES, AND WHAT `km.inertia_` REPORTS

A PROPERTY, NOT AN ACCIDENT

J is **non-increasing in k** — refining a partition cannot increase the distance to the assigned centre — and $J(m) = 0$ exactly, one cluster per point. So "minimise inertia" picks $k = m$ on any dataset; the elbow rule exists to work around that.

✓ In practice: inertia compares two runs at the *same* k — which restart was best — never two values of k.

Inertia cannot choose k: find the elbow



Inertia — the sum of squared distances to the assigned centroid — can only fall as k rises, and is zero at one cluster per point. A smooth convex curve: the *kneedle* rule — furthest below the chord, both axes rescaled — picks $k = 19$, the archive has 40, and 46% of the inertia at $k = 2$ is still present at $k = 40$.

Definition: the silhouette of a point i in cluster C_i

$$a_i = \frac{1}{|C_i| - 1} \sum_{j \in C_i,\, j \neq i} d\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right)$$

$$b_i = \min_{C \neq C_i} \frac{1}{|C|} \sum_{j \in C} d\left(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}\right)$$

- a_i: mean distance from i to the *other members of its own cluster* — how tightly it sits
- b_i: mean distance from i to the members of the *nearest other cluster*, the one with the smallest such mean
- $s_i = (b_i - a_i) \,/\, \max(a_i, b_i) \in [-1, 1]$: near $+1$ far inside its own cluster, near 0 on a boundary, negative when closer to another cluster. The **silhouette score** is its mean over all points: an interior maximum in k, and no labels needed

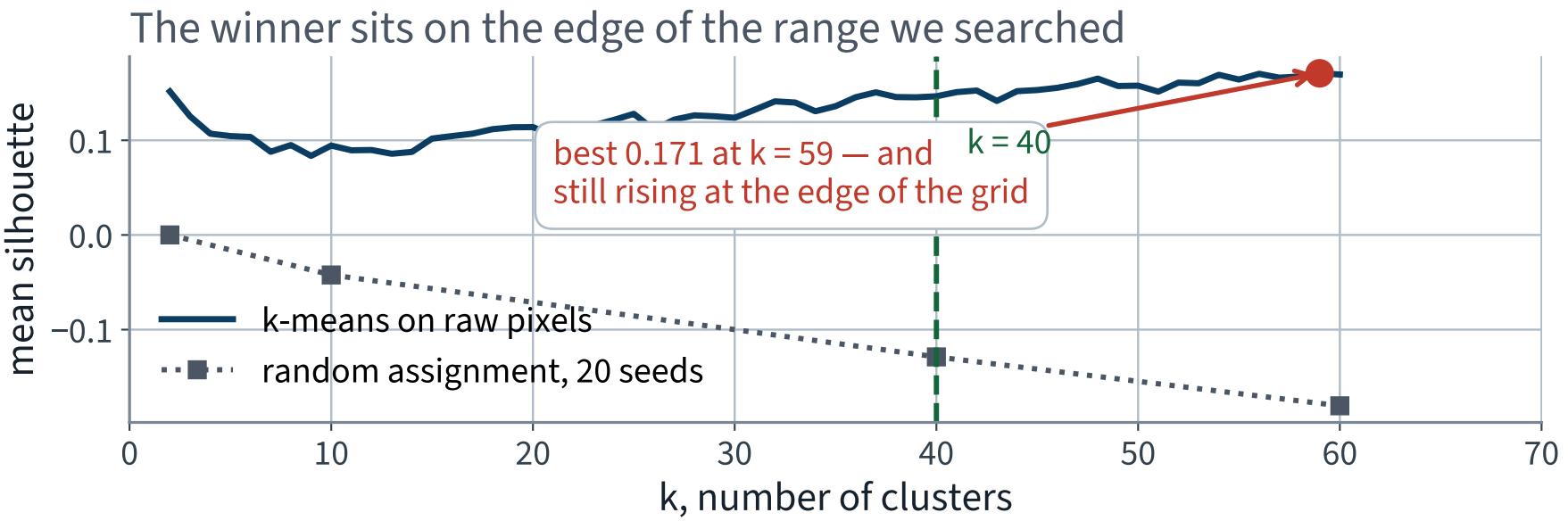
Before reading the silhouette: what does *nothing* score?

Put every photograph in a uniformly random group, twenty times, and look at the spread. The silhouette has no natural scale, and a number with no null has no units.

Random assignment, twenty times	Silhouette
$k = 10$	−0.0422 ± 0.0045
$k = 40$	−0.1288 ± 0.0107
$k = 60$	−0.1805 ± 0.0124

✓ The anchor sits at or below zero, and now you know the width of the noise around it. A silhouette of 0.02 is not a discovery.

Silhouette against k, read against the anchor



A maximum exists. It is also low, and the curve near it is nearly flat — so the chosen k is not decisively better than its neighbours. In practice that flatness is the finding: the data does not insist on a k, so the business gets to choose one.

A mean silhouette means nothing until you look

cleanest cluster at k = 40: 10 photographs, 1 identity, purity 1.00



worst cluster at k = 40: 24 photographs, 10 identities, purity 0.21



✗ **The cleanest and the worst cluster at $k = 40$. The bottom row is not a person. It is a *lighting condition* — several different people lit and posed the same way.**

Definition: purity

For one cluster C, purity is the share of its members that belong to its most common identity:

$$\text{purity}(C) = \max_{y} \frac{|\{\, i \in C : y_i = y \,\}|}{|C|}$$

- 1.00 when every photograph in the cluster is the same person; low when the cluster mixes many people with none dominating
- **Mean purity** averages it over the clusters, each cluster counting once
- It needs the identity of every photograph, so here it is an **audit** — a number the project itself could not compute

✗ Read it beside the cluster sizes: a cluster of one photograph is pure by construction, and a very large cluster can be pure only if it swallowed one person entirely. Purity alone rewards over-splitting, exactly as inertia does.

Counting it, rather than describing it

At $k = 40$, against the hidden identities (audit)	Value
Cleanest cluster: 10 photographs, purity	**1.00 ✓**
Worst cluster: 24 photographs of 10 people, purity	**✗ 0.21**
Mean purity across the forty clusters	0.74
Clusters holding exactly one person	16
Clusters with exactly ten members	7
Cluster sizes, smallest to largest	4 to 25

Every true group has exactly ten members. The overlap the distance histogram showed before anything was fitted is now a count. In practice you will not have this table — so look at a sample of every cluster, the worst ones first, before anyone names them.

Put the two halves together: compress, then cluster

```
Z = PCA(n_components=0.95, random_state=42).fit_transform(X)   # 123

for k in range(2, 61):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Z)
    sil.append(silhouette_score(Z, km.labels_))
```

The clustering code is unchanged. The grid is unchanged, the seed is unchanged, `n_init` is unchanged. Only *what k-means is looking at* changed: 123 numbers per photograph instead of 4,096.

✓ This is the standard pipeline in practice — reduce, then cluster — and the silhouette is computed in the reduced space, so it is not comparable with the one measured in pixels. A different representation is a different evaluation.

DBSCAN, in three definitions

- A point is a **core point** if at least `min_samples` points lie within `eps` of it
- Core points within `eps` of each other are in the same cluster; so are the non-core points in their neighbourhoods
- Everything else is **noise**, labelled -1

```
db = DBSCAN(eps=6.0, min_samples=3).fit(Z)
n_clusters = len(set(db.labels_) - {-1})     # exclude the noise label
```

✓ In practice: the noise label is the output you wanted for anomalies and the output you did not want for segments; `eps` is in the units of the space you hand it, so it changes when the scaling does; and `DBSCAN` has no `predict` — to label a new point you fit again, or train a classifier on its output.

Anomaly detection with the tools already on the table

By reconstruction error

An image the eigenfaces cannot rebuild is unlike the images that built them. Fit the PCA on the clean corpus, score every image, flag the worst few.

```
R   = pca.inverse_transform(pca.transform(Xa)
err = ((R - Xa) ** 2).mean(axis=1)
```

By density

DBSCAN's noise points are the images no dense region claims. That is the transactions example again, on faces.

```
bad = DBSCAN(eps=6.0, min_samples=3).fit(Z).l
```

The notebook tests the first by planting twelve corrupted images — four rotated, four dimmed, four double-exposed — and counting how many land in the top twelve, with the detector fitted on the clean corpus only.

✓ Look out for: a detector fitted on data that already contains the anomalies has learned them; and the threshold — how many to flag — is a decision about the cost of a missed one against the cost of a false alarm, not a number the method knows.

Finally: the forty labels

Split the corpus 280 / 120, stratified by identity. We may label forty of the 280 training photographs. Where we spend them turns out to matter more than what we do with them.

1. Forty **at random**
2. Forty chosen as the photograph nearest each of forty cluster centroids — `km.transform(Z).argmin(axis=0)`
3. The representative's label **propagated** to its whole cluster — 280 labels from 40
4. Propagated only to the 75% of each cluster closest to its centroid
5. All 280 true labels — the ceiling, which the project cannot buy

Same classifier, same held-out faces, twenty splits. The only thing that varies is *which* forty were labelled.

The numbers

Labels used	Held-out accuracy	sd over 20 splits
40 at random	48.1%	3.8%
40, one per cluster	**60.5% ✓**	4.8%
Propagated to the whole cluster	55.6%	4.8%
Propagated to the closest 75%	55.1%	4.9%
All 280 true labels	**✓ 97.4%**	1.6%

Twenty splits, because the differences between the middle three rows are of the same order as their spread — and a comparison that turns on less than the spread is not a comparison.

Propagation buys 280 labels from 40 and does *worse* than the 40 alone, because a wrong propagated label is worse than no label. Report that, do not bury it — it is the commonest shortcut in a labelling budget, and it is a loss.

In practice, look out for — before fitting, and with PCA

Situation	What goes wrong	Do this
Columns in different units	distance is owned by the largest unit; the segments are “spend”	scale before any distance, and keep the scaler with the model
Something dominates the variation	the clusters and the leading components are that thing — a lamp, a season, a device	measure first whether distance tracks what you care about; change the representation, not the algorithm
`n_components` as an integer or a float	a count or a variance target, one character apart	read it aloud in every pipeline you inherit
The two-dimensional plot	a projection; distances on it are not distances in the data	use it to look, never to measure

In practice, look out for — with clusters and anomalies

Situation	What goes wrong	Do this
Nobody told you k	inertia always prefers more; the silhouette is flat near its maximum	read it against a random anchor; then let the consumer choose among the flat candidates
A clean mean silhouette	one cluster can be a lighting condition, a region, a device	look at a sample of every cluster, worst first, before naming any
The job reruns next month	different seed, different segments, different names	fix `n_init` and `random_state`; check stability across resamples before trusting a segment
DBSCAN after rescaling	`eps` was set in the old units	treat `eps` as part of the representation; and remember it cannot `predict`
Anomalies in the training data	the detector has learned them	fit on data believed clean; the threshold is a cost decision
A labelling budget	propagated labels cost more than they buy	spend labels on representatives; report the loss, do not bury it

The notebook for this lecture

Open Lecture 6 in Colab

- **Runs on free CPU**, top to bottom
- PCA by hand from the SVD of the centred data, checked against scikit-learn to 10^{-4}; the explained variance ratio, the eigenfaces and the reconstructions
- Everything else on today's slides — the distance histogram, the sweep in raw pixels and compressed, the elbow, the random anchor, the silhouette diagrams, DBSCAN, the anomalies and the forty labels

> **WHAT TO CHANGE, AND WHAT SHOULD HAPPEN**
>
> Every specification box carries a **try** line. The cheapest one: change `n_components=0.95` to `0.99`, which takes d from 123 to 260. The sweep slows by roughly the ratio of the two, and what k-means finds barely moves — which is the whole trade-off in one edit.

Design exercise · in the manner of an ML system-design interview

The task: a grocery-delivery app's customer segments for its marketing team

> **THE BRIEF**
>
> A grocery-delivery app holds every account's order history. Its marketing team wants a handful of **customer segments** — one campaign per segment, written by a person and sent through the app — and every account's segment. The business reads retention and promotion spend per order; a campaign aimed at a segment that is not one wastes both. No account comes with a label.

- **Given:** for each account, its orders over the last year — what, when, how much, on promotion — and nothing else
- **Scale:** millions of accounts; dozens of departments; a budget to interview a few hundred customers
- **Consumers of the output:** the marketing team, who name and write to each segment; the campaign tool assigning new accounts; finance

✓ Nine steps follow, each with the pitfall that catches a first attempt and what to do about it. The model at the centre is today's: PCA to reduce the dimension, then k-means, with k chosen without labels.

Step 1 — Frame it honestly: there is no target column

- One row per account; the output is a segment for every account and a *profile* of each segment in the measured columns
- No truth to compare with, so the metric compares the grouping to itself — the silhouette, read against what a **random** grouping scores — and to its own stability under resampling
- The baseline is “group at random”; the test is whether a marketer reading a segment’s baskets sees one kind of customer

> **PITFALL**
>
> The first attempt asks the model how many kinds of customer there are, reads it as fact, and writes a campaign per cluster. Every number here is a statement about *geometry*; whether the geometry is the customers is answered by looking. Decide before fitting what to look at: the baskets nearest each centre, and the worst segment as well as the best.

Step 2 — The row, scaled before any distance is measured

- One account is a vector of *shares and rates* — the share of spend per department, orders per week, basket size, the share on promotion — so that how much a household spends is not an axis of its own
- k-means groups rows close in Euclidean distance, which adds every column up with the same weight: standardise, with a scaler fitted on the training accounts, before the first distance

> **PITFALL**
>
> Annual spend in euros next to orders per week: the column with the largest unit decides every distance, and the "segments" are bands of total spend — a lamp and a nose on one axis, a campaign finance could have written from one column. Remove the nuisance at the row, not by deleting leading components afterwards: that threw away signal with the nuisance today.

Step 3 — PCA, and how many components

- Dozens of correlated department columns: centre, take the SVD, and let a float to `n_components` choose d from the cumulative variance ratio — an integer would say how many
- A component is a direction in the columns, so it can be read: fresh against packaged, evening top-ups against one weekly shop
- Both cost models below are linear in n, so the sweep over k and the silhouette get cheaper by the same factor

> **PITFALL**
>
> Variance kept is not usefulness kept. The first component of a table of departments is usually “how much” — total spend wearing forty names — which the shares and rates of the previous step were meant to remove. Read every kept component in the columns before clustering on it; if the first one is size, the scaling is wrong, and no choice of d repairs it.

Step 4 — The model: k-means, and what it assumes whether you asked or not

- Lloyd's algorithm from k-means++ starts; name `n_init` and keep the best of several restarts, because it converges to a local minimum that depends on the start
- It assumes convex, spherical, equal-sized clusters, total assignment and a given k — and the app's customers are one large crowd with a few small, distinct groups around it

> **PITFALL**
>
> Voronoi cells slice the crowd into arbitrary wedges and absorb the small groups, and every account, however odd, gets one. Read the cluster sizes and the silhouette diagram before believing the partition. If the shape is wrong, a Gaussian mixture fits ellipsoids of different sizes; DBSCAN finds dense groups and labels the rest noise, at the price of no `predict` for a new account.

Step 5 — Choose k: inertia cannot, the silhouette can, against a random anchor

- Inertia is non-increasing in k and zero at one cluster per account: no interior minimum exists, and the elbow is a picture of a criterion that does not exist
- The silhouette is penalised for splitting a group and for merging two, so it has an interior maximum; quadratic in the rows, it is computed on a sample of the accounts, in the reduced space
- Before reading any value, assign the accounts to k groups at random, many times: the null, with its width

PITFALL

A silhouette with no anchor has no units, and the marketing team is told that “the data shows five segments” — and budgets five campaigns — on a maximum that is low and nearly flat across its neighbours. Report the curve, the anchor and the width of the noise, and say when the chosen k is not decisively better than its neighbours.

Step 6 — Do the segments survive a resample?

- Refit at the chosen k on several random subsamples, match the segments of two fits by overlap, and count the share of the accounts they share that keep their segment
- A segment the team can write to is one that is there again after a resample; one that moves on every refit is a property of the sample, not of the customers

PITFALL

The silhouette picks a k, the campaign is written, and the next refit — a new quarter, a different seed — produces segments that do not match: a message written for one segment reaches accounts no longer in it. Restarts remove the variation that is Lloyd's algorithm; what remains is the data's. Report the index between refits beside the silhouette, and treat a k whose partitions disagree as unsupported, whatever the silhouette says.

Step 7 — Name a segment without inventing a story

- For each segment: its profile in the *original* units, every column's mean against the mean of all accounts, and the baskets of the accounts nearest its centroid
- Spend the interview budget on those representatives, not on accounts drawn at random: today representatives were worth a quarter more than random, with the same classifier
- Show the worst segment as well as the cleanest

> **PITFALL**
>
> "Young professionals" is written on a centroid nobody looked at, and a campaign goes out in that voice; the segment is actually "accounts delivering to an office at lunchtime" — a *condition*, this problem's lighting, not a kind of person. Name a segment only in the measured columns, and never propagate an interviewed answer to its whole segment: propagated labels did worse than the few alone.

Step 8 — In production: the accounts that fit nowhere, and the seasons

- k-means assigns every account, so score each one first — the density under a Gaussian mixture, or the PCA reconstruction error: the lowest few are a shared flat's account or a café buying for resale, and get no segment
- Release with a holdout of accounts that get no campaign, so each campaign's lift on retention is measured; refit on a rolling window each season, compare with the old partition by the share of accounts that keep their segment; the data team is on call

> **PITFALL**
>
> Fitted in summer, run in the winter holidays: the scaler, the components and the centres describe a season that has gone, and the share of accounts that change segment on refit is the only sign. Monitor that share and the column means; when they move, refit, and tell the team which segments survived.

Step 9 — Ship it: the pipeline, and the pitfalls in one table

Order log → one row per account, shares and rates → held-out accounts → scaler and PCA in a pipeline → k-means with restarts → silhouette against the random anchor → stability across resamples → profiles and representatives → anomaly scores → a holdout; refit by season.

Pitfall	Address it by
Geometry read as people	deciding what will be looked at before fitting
The largest unit chooses the segments	shares and rates; a scaler fitted on the training accounts
Variance kept read as usefulness	reading every kept component in the columns
Equal convex cells on an unequal crowd	sizes and the silhouette diagram; a mixture or DBSCAN
The elbow, or a silhouette with no units	the silhouette against a random anchor, with its noise
Segments that move on every refit	the share of accounts that keep their segment across resamples

What to remember

1. Unsupervised learning finds groups, cuts the numbers per item, and flags the unusual — with no label to score against, so every number is about geometry and you find out you were wrong by looking
2. The tool follows the shape of the answer: every item in a compact group → k-means; dense regions of any shape and a “none of these” → DBSCAN; fewer numbers with the price stated → PCA; themes in sparse counts → the truncated SVD. Each is the wrong tool for the other three jobs
3. PCA is the SVD of the **centred** data; the error it leaves is the tail of the spectrum, and the explained variance ratio reads it before you choose d: 123 components hold 95% of 4,096 pixels
4. Inertia cannot choose k. The silhouette can, read against a random anchor — and then against the items themselves, because a mean hides a cluster that is a lighting condition rather than a person
5. Scale before any distance; fix the seed; look at the worst cluster first; fit a detector on clean data; the threshold and k are decisions the consumer owns
6. Forty labels spent on cluster representatives were worth a quarter more accuracy than forty spent at random; propagating them was a loss. Which forty mattered more than what was done with them
7. Designed a grocery-delivery app’s customer segments for its marketing team in nine steps — shares and rates scaled before any distance, PCA read in the columns, k-means read against its assumptions, k by silhouette against a random anchor, stability across resamples, names in the measured columns, anomalies by density, a holdout and a refit by season — each step named for the pitfall it avoids

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 6

five, in the style of the written paper

Exercises — Lecture 6

1 A marketing team wants every customer placed in one of a handful of segments; a fraud team wants the few transactions unlike all the rest, with no labels yet. Name the clustering method for each, and say why the other one is the wrong tool in each case. [5]

2 The explained variance ratio is sometimes called a heuristic for choosing d. Say why it is not one, and what keeping 95% of the variance leaves as reconstruction error. [5]

3 A customer-segmentation job reruns every month on fresh orders, and the marketing team complains that the segments "moved": the groups and their sizes differ from last month's. Name two things to fix in the code, and one check to run before any segment is trusted. [4]

Solutions on Lecture 7's deck.

Exercises — Lecture 6 (continued)

4 Inertia always falls as k rises, so it cannot choose k. State what silhouette adds, and one situation where it also fails. [5]

5 You have 4,096-dimensional face vectors and are told two faces are “close”. Explain why that alone does not mean they are the same person. [4]

Solutions on Lecture 7’s deck.

THE FIVE SET AT THE END OF LECTURE 5

Solutions Lecture 5

not part of this lecture

Solutions — Lecture 5

1. For n predictors each of variance σ^2 and pairwise correlation ρ, the variance of their average…

✓ It tends to $\rho\sigma^2$ — correlation, not ensemble size, sets the floor.

- The second term vanishes; the first does not depend on n at all
- Adding members past a point buys nothing, which is why the design effort goes into *decorrelating* them

2. Bagging measures $\rho = 0.072$ with one member at 78.8% accuracy; the random forest measures $\rho = 0.043$…

✓ Lowering ρ is not free. The forest lowers it by letting each split see only 7 of 54 columns, which handicaps every member, and the formula is about variance alone: σ^2 sits in both terms and bias is not in it at all.

- $\mathrm{Var}(\bar{f}) = \rho\sigma^2 + (1-\rho)\sigma^2/n$ predicts the variance of the average; accuracy also depends on how good each member is
- On this dataset the decorrelation is worth less than the weaker members cost: 89.8% for bagging against 88.4% for the forest

Solutions — Lecture 5 (2)

3. You set bootstrap=False and oob_score=True. State what happens and why.

✓ It raises: with no bootstrap there are no out-of-bag rows.

- Out-of-bag scoring uses the rows a member did not sample
- Without sampling, every member sees every row and the estimate is undefined

4. Impurity importance ranks a column of random numbers tenth of 55; permutation importance on held-out data…

✓ Shuffle one column of the held-out set, re-score the fitted model, and report the fall in accuracy, repeated for a spread. A random column carries no information the model can use on new rows, so shuffling it costs nothing.

- Impurity importance sums the impurity decrease of every training-set split on a column, so it is earned on the rows the trees were grown on
- Permutation importance is measured on rows the model never saw: only a column whose values matter for those predictions loses accuracy when they are scrambled

Solutions — Lecture 5 (3)

5. A forest improves accuracy over one tree by far less than the variance reduction would suggest. Is this a…

✓ No — variance is only one term of the error.

- Averaging reduces variance and leaves bias untouched
- If the remaining error is mostly bias or noise, removing variance moves the accuracy very little

LECTURE 7 · GÉRON CH. 9

Neural networks

Back to labels: ten classes of small greyscale images, and a pixel as a feature
The multilayer perceptron, and why a stack of linear layers is a linear model
A layer in matrices, softmax and cross-entropy, and the parameter count before the fit
A network fitted and tuned by hand — and the four walls `fit()` puts round you

Run the Lecture 6 notebook first